

KSRTC VTMS — Vehicle Tracking & Management System

Proof of Concept Documentation

Prepared by: MongoDB Solutions Architecture
Version: 1.0
Date: May 2026
GitHub: <https://github.com/sumitJha-Realm/ksrtc-vtms-poc>
Live Demo: <https://ksrtc-vtms-poc.vercel.app>

Table of Contents

- 1. [Executive Summary](#)
- 2. [Solution Architecture](#)
- 3. [Data Model](#)
- 4. [MongoDB Features Demonstrated](#)
- 5. [How to Use the Application](#)
- 6. [API Reference](#)
- 7. [Setup & Installation](#)
- 8. [Future Enhancements](#)
- 9. [Tech Stack](#)

1. Executive Summary

The **KSRTC VTMS PoC** is a full-stack demonstration of how **MongoDB Atlas** can power a real-time Vehicle Tracking & Management System for KSRTC (Karnataka State Road Transport Corporation) — supporting **8,000+ buses**, **10-second GPS intervals**, **real-time alerts**, and **operational analytics**.

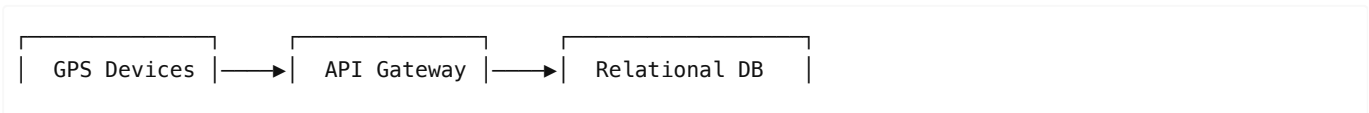
This PoC validates that MongoDB Atlas can serve as a **unified platform** eliminating the need for separate databases, message queues, or geospatial engines — handling high-velocity telemetry, real-time event processing, and complex analytics in a single system.

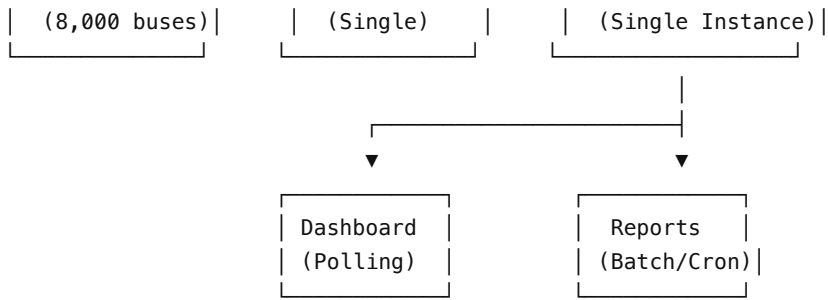
Key Capabilities Demonstrated

Capability	Description
Real-time Tracking	50 simulated buses updating every 10 seconds on a live map
Change Stream Alerts	Automatic alert generation for overspeed, geofence breach, harsh braking
Route Playback	Historical GPS trail replay with speed-coded animations
Fleet Analytics	Aggregation pipeline reports with detection logic transparency
Geospatial Processing	Point-in-polygon, nearest stop, route corridor deviation
Time Series Optimization	Bucketed GPS data with TTL auto-expiry

2. Solution Architecture

Current Architecture (Assumed at KSRTC)

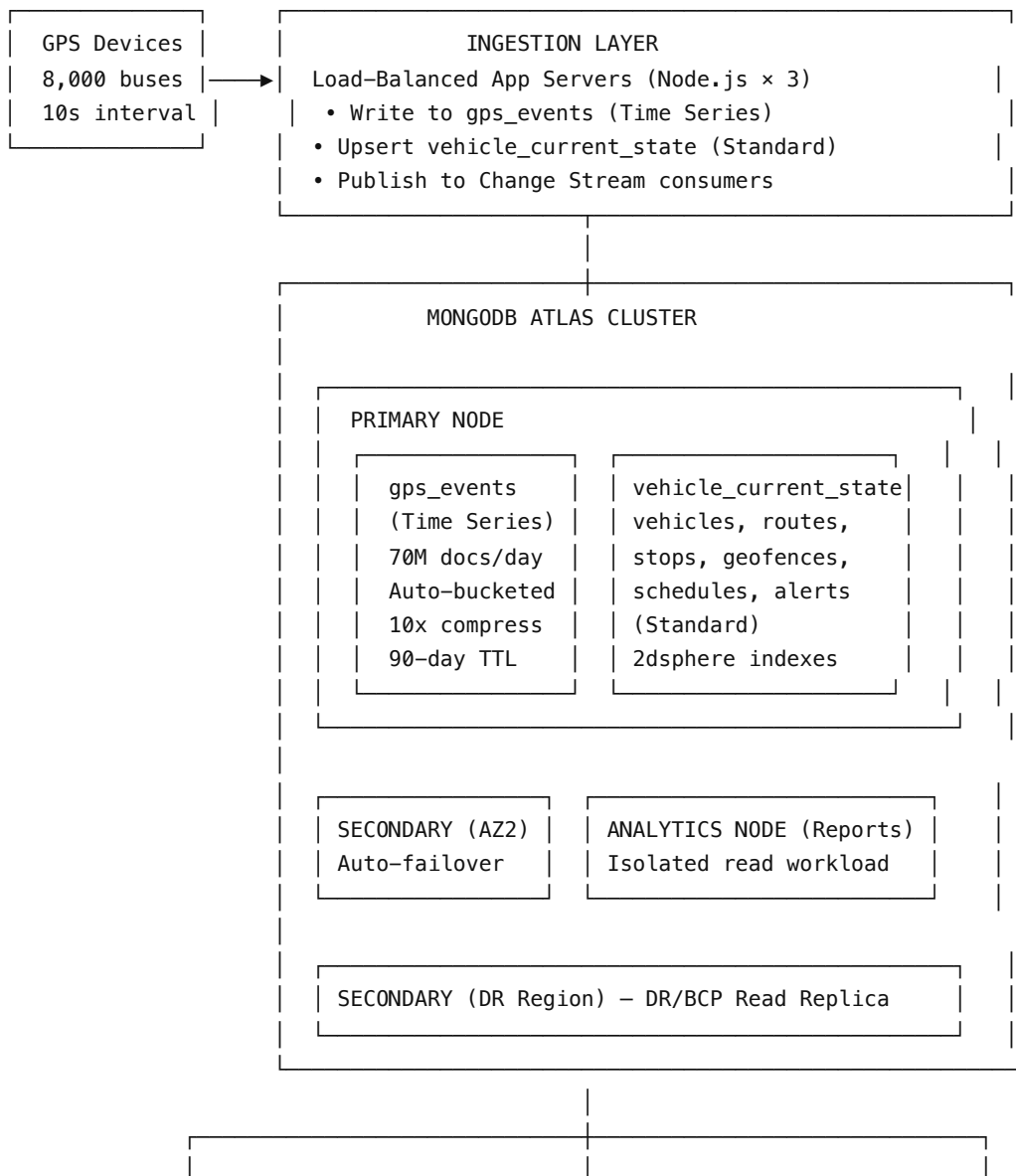


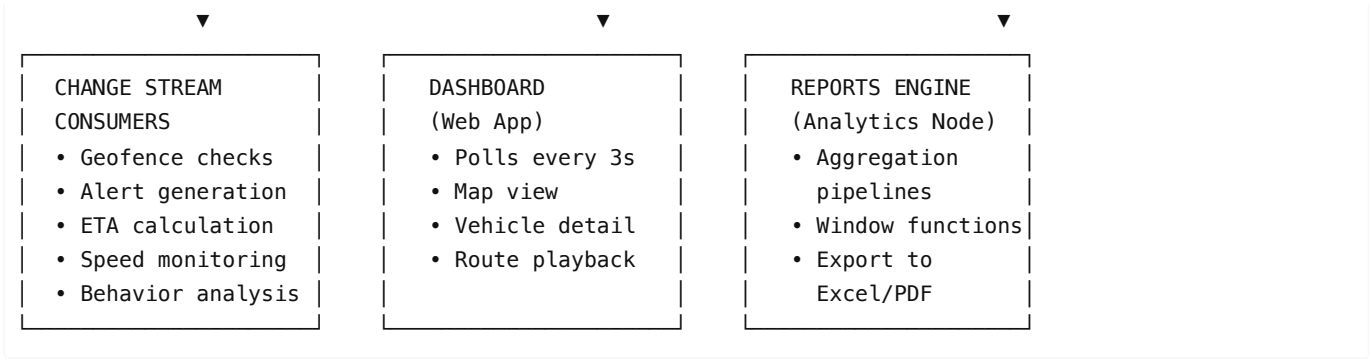


Problems:

- Single point of failure (SPOF)
- No horizontal scaling for write throughput
- GPS data and operational data in same tables – contention
- No real-time event processing
- Reports block operational queries
- No built-in geospatial support
- Manual data archival needed

Proposed Architecture (MongoDB Atlas)





Data Flow — Time Series vs Standard

Aspect	Time Series Collection	Standard Collection
Data Nature	Machine-generated, continuous, immutable	Human/system managed, mutable
Write Pattern	Insert-only, never update	Insert + frequent updates
Volume	70 million/day (99.5% of all data)	~200K/day (0.5%)
Query Pattern	Always time-range + vehicle filter	Any field: ID, status, location
Retention	Auto-expire after 90 days (TTL)	Kept indefinitely
Optimization	Auto bucketing, columnar compression	B-tree + 2dsphere indexes

3. Data Model

Collections

Collection	Type	Purpose	Daily Volume
gps_events	Time Series	Every GPS ping — coordinates, speed, heading	70M
vehicle_current_state	Standard	Latest position/status per vehicle (upserted every 10s)	8K upserts/10s
vehicles	Standard	Bus master — registration, make, model, depot	—
routes	Standard	Route definitions — GeoJSON LineString + ordered stops	—
bus_stops	Standard	Stop master — GeoJSON Point, associated routes	—
geofences	Standard	Depot boundaries, school zones — GeoJSON Polygon	—
schedules	Standard	Trip schedules — vehicle x route x expected times	—
alerts	Standard	All events — overspeed, panic, geofence violations	~100K
depots	Standard	Depot/division metadata	—

Key Indexes

```
// GPS events — compound for playback queries
{ "metadata.vehicleId": 1, "timestamp": -1 }

// Geospatial — for $geoWithin / $geoNear
{ "location": "2dsphere" }
```

```
// Alerts – status + time for active alert queries
{ "status": 1, "timestamp": -1 }

// Vehicle current state – fast lookup
{ "vehicleId": 1 }
```

Demo Data (PoC)

Entity	Count	Notes
Vehicles	50	Karnataka registration numbers (KA-01-F-XXXX)
Routes	10	Real Bangalore routes (Majestic→Electronic City, etc.)
Stops	80	8 stops per route with real coordinates
Depots	10	Bangalore depots (Kempegowda, Jayanagar, Peenya, etc.)
Geofences	10	Depot zones + restricted areas

4. MongoDB Features Demonstrated

Feature	Usage in VTMS
Change Streams	Real-time alert generation when vehicle state changes (overspeed, geofence breach)
Geospatial Queries	\$geoWithin, \$geoNear for geofence detection, nearest-stop calculations
Aggregation Pipelines	Fleet analytics, depot summaries, speed reports, trip efficiency
2dsphere Indexes	Spatial indexing on GPS coordinates for fast geospatial lookups
TTL Indexes	Auto-expire old GPS events (configurable retention)
Compound Indexes	Optimized queries on vehicleId + timestamp for playback
\$lookup	Join vehicles with routes, depots, and stops for enriched views
Time Series Collections	Purpose-built storage for high-velocity GPS telemetry
\$setWindowFields	Harsh braking, unauthorized stop, signal gap detection
Bucket Pattern	Time-series GPS data organization

Window Functions — Key Detections

Detection	Technique	Threshold
Harsh Braking/Acceleration	\$shift to compare speed at T vs T-10s	±4 m/s²
Unauthorized Stop	Sliding window sum of stationary pings	18 consecutive pings (3 min)
Signal Gap	\$dateDiff between consecutive pings	> 30 seconds
Sustained Overspeed	\$avg over 30-second window	Average exceeds limit
Breakdown Inference	\$avg + \$max speed = 0 for 5+ minutes	After prior movement
ETA Prediction	\$expMovingAvg(N: 10)	Dynamic recalculation

5. How to Use the Application

5.1 Live Dashboard (Home Page)

URL: `/`

1. **View all vehicles** — All 50 buses appear on the map as colored markers (green = running, orange = idle, red = stopped)
2. **Click a vehicle marker** — Popup shows vehicle ID, speed, route, depot, heading, last update
3. **Side panel stats** — Fleet-wide statistics: total vehicles, running/idle/stopped counts, alerts
4. **Auto-refresh** — Dashboard polls every 5 seconds for new positions
5. **Speed-based colors** — Markers change based on speed; pulsing red indicates overspeed

MongoDB Feature: Queries `vehicle_current_state` — updated in real-time by Change Streams.

5.2 Route Playback

URL: `/playback`

1. **Select a vehicle** from the dropdown (e.g., KA-19-F-1001)
2. **Pick a time range** — "Last 1 Hour", "Last 6 Hours", or custom dates
3. **Click "Load Trip"** — GPS trail loads as a color-coded polyline
4. **Watch playback** — Bus marker moves along trail with heading rotation
5. **Use controls** — Pause/Play, speed up (2x, 4x, 8x), drag progress slider
6. **Speed coloring** — Green (normal), orange (elevated), red (overspeeding)

MongoDB Feature: Queries `gps_events` Time Series with bucket-optimized range scans.

5.3 Alerts

URL: `/alerts-page`

1. **Browse alerts** — Left panel shows all alerts sorted by time with severity indicators
2. **Click any alert** — Right panel shows:
 - GPS trail map (± 2 minutes around alert, color-coded by speed)
 - Pulsing red marker at exact alert trigger point
 - Speed graph (bar chart over ± 2 min window)
 - Calculation box showing detection formula
3. **Read explanation** — Human-readable description of what happened
4. **Acknowledge/Close** — Update alert status
5. **Filter by type** — Dropdown filters for overspeed, harsh braking, geofence, etc.

MongoDB Features: Change Streams for generation, `$setWindowFields` with `$shift` for context.

5.4 Reports

URL: `/reports`

Available reports:

- **Vehicle Tracking Summary** — distance, hours, speed per vehicle
- **Overspeeding Violations** — all overspeed events with context
- **Harsh Braking/Acceleration** — window function detection
- **Unauthorized Stops** — sliding window stop detection
- **Schedule Adherence** — actual vs expected timing

Each report includes a **"Show Logic"** button explaining the aggregation pipeline used.

MongoDB Feature: Complex aggregation pipelines with `$group`, `$lookup`, `$setWindowFields`.

5.5 Quick Start Workflow

Step	Action	Result
1	Open app at http://localhost:3000	Live Dashboard loads
2	Select hours (2h–48h) → Click 🌱 Seed Data	Historical GPS events generated
3	Click ▶ Simulator	Buses start moving in real-time
4	Navigate to Alerts, Playback, Reports	Explore MongoDB features

Seed Data Estimates:

- 2h → ~36K GPS events, 6 alerts
- 6h → ~108K GPS events, 18 alerts
- 24h → ~432K GPS events, 72 alerts

6. API Reference

Method	Endpoint	Description
GET	/api/dashboard/summary	Fleet summary (running/idle/stopped, depot breakdown)
GET	/api/dashboard/vehicles	All vehicle current positions
GET	/api/playback/:vehicleId	GPS trail for time range
GET	/api/alerts	List alerts (filterable by status/type)
GET	/api/alerts/:alertId/context	Alert + surrounding GPS trail
GET	/api/reports/fleet-speed	Speed analytics aggregation
GET	/api/reports/trip-efficiency	Trip efficiency metrics
GET	/api/reports/distance-report	Distance covered per vehicle
GET	/api/reports/event-context	Report event with GPS context
GET	/api/vehicles/routes/list	All routes with geometry
GET	/api/vehicles/routes/:routeId	Route detail + stops
GET	/api/geofence/zones	All geofence polygons
POST	/api/admin/seed-demo?hours=6	Generate historical data
POST	/api/admin/simulator/start	Start live GPS generation
POST	/api/admin/simulator/stop	Stop GPS generation
GET	/api/admin/simulator/status	Check simulator state

7. Setup & Installation

Prerequisites

- **Node.js** 18+
- **MongoDB Atlas** cluster (M0 free tier works for demo)

- npm

Installation Steps

```
# 1. Clone the repository
git clone https://github.com/sumitJha-Realm/ksrtc-vtms-poc.git
cd ksrtc-vtms-poc

# 2. Install dependencies
npm install

# 3. Configure environment
cp .env.example .env
# Edit .env with your MongoDB Atlas connection string:
# MONGODB_URI=mongodb+srv://<user>:<pass>@<cluster>.mongodb.net/?retryWrites=true&w=majority
# DB_NAME=ksrtc_vtms

# 4. Seed reference data (routes, stops, vehicles, depots, geofences)
node src/seed/seedData.js

# 5. Start server
node server.js

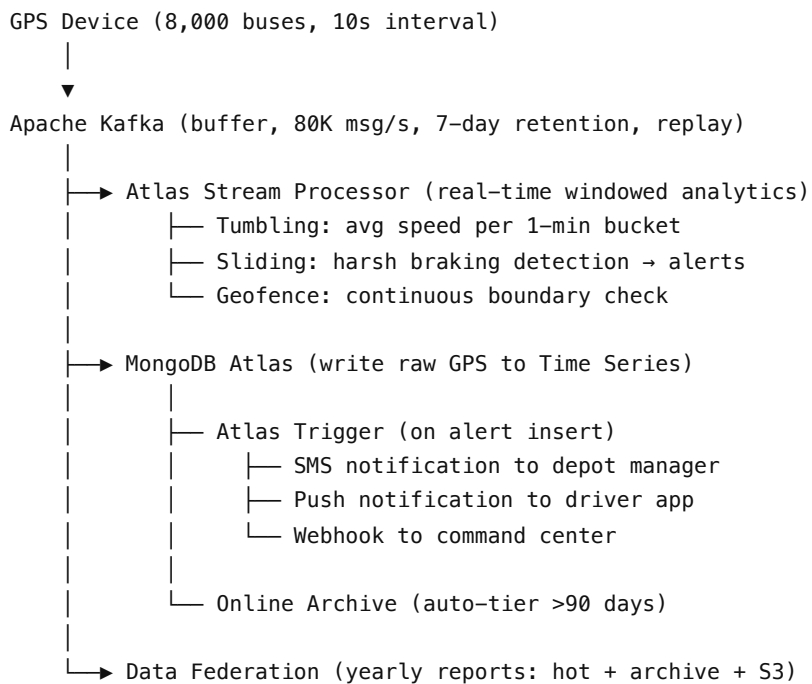
# 6. Open browser
# Navigate to http://localhost:3000
```

Project Structure

```
ksrtc-vtms-poc/
├── server.js                # Express server entry point
├── package.json
├── .env.example             # Environment variables template
├── src/
│   ├── config/database.js   # MongoDB Atlas connection (pooling, retry)
│   ├── changeStreams/gpsProcessor.js # Change Stream processor
│   ├── models/setupCollections.js # Collection schemas, indexes
│   ├── routes/
│   │   ├── admin.js         # Seed data + GPS simulator
│   │   ├── alerts.js        # Alert CRUD + context
│   │   ├── dashboard.js     # Fleet summary, positions
│   │   ├── geofence.js      # Geofence CRUD + detection
│   │   ├── playback.js      # Historical GPS playback
│   │   ├── reports.js       # Aggregation-based reports
│   │   └── vehicles.js      # Vehicle + route queries
│   └── seed/
│       ├── seedData.js      # Initial data seeder
│       └── gpsSimulator.js   # GPS simulator (CLI)
└── public/
    ├── index.html           # Live Dashboard
    ├── css/style.css        # Dark theme styles
    ├── js/app.js            # Client logic (Leaflet, polling)
    └── pages/               # Alerts, Playback, Reports, etc.
```

8. Future Enhancements

Enhanced Production Architecture



8.1 Apache Kafka — Ingest Buffer

Aspect	Current PoC	With Kafka
Data loss on DB downtime	Yes — messages dropped	No — buffered in Kafka
Max ingestion rate	~5K/s (Express bottleneck)	80K+/s (partitioned)
Multiple consumers	Not possible	Analytics, Archival, Alerts in parallel
Reprocessing	Impossible once consumed	Replay from any offset

Benefits: Zero data loss, 80K msg/sec throughput, 7-day retention, replay capability, back-pressure handling.

8.2 Atlas Triggers — Serverless Event Processing

Aspect	Current (Change Stream)	Atlas Triggers
Availability	Tied to app server uptime	99.995% (Atlas managed)
Scaling	Manual — add more servers	Automatic — event-driven
Retry on failure	Must implement manually	Built-in with configurable retries
External integration	Add HTTP client to app	Native HTTP service in functions
Cost	Always-on server required	Pay-per-execution (serverless)

8.3 Atlas Stream Processing — Real-Time Analytics

Process, aggregate, and transform streaming GPS data using MongoDB's native stream processor with MQL syntax.

- **Tumbling Windows:** Avg speed per 1-min bucket per vehicle
- **Sliding Windows:** Harsh braking detection in real-time
- **Native MQL:** Same MongoDB query language — no Spark/Flink expertise needed

- **Kafka Source/Sink:** Read from Kafka, write to MongoDB or other topics
- **Geofence Processing:** Continuous boundary checks on the stream

8.4 Online Archive — Automated Data Tiering

Data Age	Storage Tier	Cost/GB/month	Query Speed
0–90 days	Hot (SSD)	\$2.30	< 50ms
90 days – 5 years	Online Archive	\$0.02	1–5 seconds

Result: 99% cost reduction on historical data while maintaining query access.

8.5 Data Federation — Unified Cross-Source Queries

Query across Atlas clusters, Online Archive, S3 buckets, and HTTP sources with a single MQL query — enabling yearly fleet performance reports without keeping all data in expensive hot storage.

Combined Impact

Metric	Current PoC	Enhanced Production
Ingestion capacity	~5K GPS/sec	80K+ GPS/sec
Data loss risk	Possible on app/DB failure	Zero — Kafka buffers
Alert latency	~5s (Change Stream)	<100ms (Stream Processor)
Notification delivery	None (manual check)	Auto SMS/Push (Triggers)
Storage cost (5 years)	~\$200/month (all hot)	~\$25/month (tiered)
Historical queries	Only recent data	Full 5-year queryable
Operational overhead	Manage server process	Fully managed (serverless)

9. Tech Stack

Layer	Technology
Backend	Node.js, Express.js
Database	MongoDB Atlas (6.x driver)
Frontend	Vanilla JS, Leaflet.js, CSS3
Maps	Carto Dark tiles, Leaflet polylines/markers
Geospatial	Turf.js (server-side), MongoDB 2dsphere
Real-time	Change Streams (app-level), Polling (3s)
Hosting	Vercel (serverless deployment)

10. Platform Capabilities (Production)

High Availability & DR

- 3-node replica set across 2 Availability Zones
- Automatic failover in 10-30 seconds

- Atlas SLA: 99.995% uptime guarantee
- Cross-region DR replica (asynchronous, < 1s lag)
- Point-in-time restore: 1-second granularity

Auto-Scaling

- Compute auto-scaling based on CPU/memory/IOPS thresholds
- Storage auto-scaling — grows with data
- Handles 2x peak traffic without manual intervention

Security

- Encryption at rest (AES-256) and in-transit (TLS 1.2+)
- VPC peering / Private endpoints
- Role-based access control (RBAC)
- LDAP/SSO integration
- Field-level encryption for sensitive data
- Database-level audit logging

Monitoring

- Real-time performance metrics
- Query Performance Advisor (auto-suggests indexes)
- Custom alerts (PagerDuty, Slack, email)
- Query profiler for optimization

Contact & Resources

Resource	Link
GitHub Repository	github.com/sumitJha-Realm/ksrtc-vtms-poc
Live Demo	ksrtc-vtms-poc.vercel.app
MongoDB Atlas	cloud.mongodb.com
MongoDB Documentation	docs.mongodb.com

This document is prepared for internal and external sharing as part of the MongoDB Solutions Architecture engagement with KSRTC.